

FRAMEWORKS DE IA

Cierre de Asignatura

Ingeniería Civil Informática · Electivo 4° año

ICIIT — Universidad del Desarrollo

UNIDAD 1 — Fundamentos de Frameworks

Contenidos

- Ecosistema PyTorch y TensorFlow/Keras
- Tensores, autograd y grafos computacionales
- nn.Module, DataLoader y loop de entrenamiento
- Keras APIs funcional y secuencial, callbacks
- Guardar y cargar modelos (state_dict, checkpoint, SavedModel)

Técnicas

- Clasificación de texto con BoW y embeddings
- Adaptación de arquitecturas CNN a datos no imagen (LeNet-5, VGG16)
- Entrenamiento desde cero vs modelos preentrenados

Herramientas

- torch, torch.nn, torch.optim
- tf.keras, tf.data
- torchvision.models

UNIDAD 2 — Visión Computacional y Evaluación

Contenidos

- Arquitecturas CNN: LeNet-5, VGG16, MobileNetV3
- Transfer learning y fine-tuning en dos fases
- Clases desequilibradas y estrategias de corrección
- Métricas para clasificación con imbalance
- Interpretabilidad con Grad-CAM
- Métricas de calidad de imagen en tiempo real

Técnicas

- Transfer learning con base congelada + cabezal custom
- Fine-tuning progresivo (fine_tune_at)
- Oversampling sintético (SMOTE, SMOTETomek)
- Undersampling (RandomUnderSampler, TomekLinks, NearMiss)
- Class weights y WeightedRandomSampler
- Focal Loss (γ, α) — Lin et al. 2017
- Varianza del Laplaciano para nitidez (Sharpness)
- FPS con suavizado exponencial (EMA)
- Grad-CAM con tf.GradientTape

Herramientas

- imbalanced-learn (SMOTE)
- tf.keras.metrics (Recall, Precision, AUC-PR, F1)
- matplotlib (matriz de confusión, curvas ROC, AUC-PR)
- OpenCV.js (procesamiento en browser)
- Ionic 8 + Angular 18 (app móvil/web)

UNIDAD 3 — MLOps y Despliegue

Contenidos

- ONNX como formato universal de modelos
- Conversión de modelos a ONNX desde PyTorch y TensorFlow
- Inferencia en producción con ONNX Runtime
- Optimización de hiperparámetros con Optuna
- Destilación de conocimiento entre arquitecturas
- Integración de modelos en aplicaciones web/móvil

Técnicas

- Exportación con `torch.onnx.export()` — tracing
- Exportación con `tf2onnx` (desde Keras y SavedModel)
- `dynamic_axes` para batch dinámico en producción
- Selección de Execution Provider (CPU, CUDA, CoreML, TensorRT)
- Búsqueda bayesiana de hiperparámetros (TPE)
- Pruning de trials ineficientes (MedianPruner)
- Destilación clásica con KL Divergence (mismo vocabulario)
- Sequence-level Knowledge Distillation (vocabulario distinto)
- Inferencia no bloqueante con `requestAnimationFrame` fuera de NgZone
- Gestión manual de memoria WASM (`cv.Mat.delete()`)

Herramientas

- `onnx`, `onnxruntime`, `onnxruntime-web`
- `tf2onnx`, `torch.onnx`
- Netron (`netron.app`) — visualizador de grafos
- Optuna (TPE, MedianPruner)
- Hugging Face Transformers (`GPT2LMHeadModel`, `GPTNeoForCausalLM`)
- Angular Signals, NgZone, `ChangeDetectionStrategy.OnPush`

PROYECTO INTEGRADOR

Pipeline IA end-to-end: entrenamiento → evaluación → exportación ONNX → despliegue en app Ionic + Angular con inferencia en tiempo real sobre frames de cámara.

ACTIVIDADES DE PROFUNDIZACIÓN

Transfer Learning con MobileNetV3

Demostró que no es necesario entrenar desde cero para obtener un producto funcional. Un modelo preentrenado en ImageNet, con las capas superiores reentrenadas sobre un dataset específico, alcanzó resultados sólidos en pocas épocas y con recursos mínimos. Esta es la práctica estándar en la industria.

Destilación de Modelos LLM

GPT-2 como profesor hacia GPT-Neo y TinyLlama como alumnos, en dos enfoques: KL Divergence sobre logits cuando el vocabulario es compartido, y sequence-level distillation cuando no lo es. Esta actividad conecta directamente con la realidad actual del campo: la mayoría de los LLMs open-source disponibles hoy fueron entrenados con datos generados por modelos más grandes.

FUNCIÓN DE UN INGENIERO I+D

1. Exploración y selección tecnológica

La comparativa entre PyTorch y TensorFlow, la elección entre arquitecturas CNN y la decisión entre técnicas de imbalance son ejercicios directos de selección tecnológica informada. No se elige por preferencia — se elige con criterio medible.

2. Experimentación y prototipado rápido

Optuna automatiza el ciclo experimental — cada trial es una hipótesis de configuración. La búsqueda bayesiana (TPE) y el pruning de trials replica el método científico: explorar, medir, descartar lo que no funciona.

3. Evaluación rigurosa de resultados

El énfasis en que accuracy es engañosa con clases desequilibradas, y la adopción de F1, AUC-PR y Recall replica la responsabilidad del ingeniero I+D de reportar resultados con honestidad técnica — incluyendo los errores del modelo.

4. Transferencia tecnológica

Convertir un modelo a ONNX, verificarlo y desplegarlo en una app Ionic es el pipeline exacto que separa un experimento académico de un producto de ingeniería.

5. Optimización de recursos

La destilación de conocimiento (8× menos parámetros) y la selección de Execution Provider en ONNX Runtime son técnicas directas de optimización. Calcular Sharpness cada N frames responde a la misma lógica: solo el cómputo necesario.

6. Documentación y comunicación técnica

Los comentarios guía en el InferenceService, el README del proyecto Ionic y los laboratorios documentados son ejercicios de comunicación técnica — el ingeniero I+D que no documenta no transfiere conocimiento.

7. Trabajo en la frontera del conocimiento

Adaptar LeNet-5 para clasificar texto con embeddings, o destilar GPT-2 hacia GPT-Neo con vocabularios distintos, son ejercicios sin receta directa. El estudiante tuvo que razonar la adaptación — eso es trabajo en la frontera.

CONCLUSIÓN

Valor agregado: leer el modelo para tomar decisiones

El objetivo central de la asignatura no fue entrenar modelos — fue desarrollar la capacidad de interpretar lo que los modelos comunican a través de sus métricas, y actuar con criterio sobre esa información.

■ Dataset con problemas

Cuando accuracy es alta pero Recall es cercano a cero, el modelo no aprendió el problema: memorizó la clase mayoritaria. La solución no está en el modelo sino en los datos. SMOTE, class weights y Focal Loss son respuestas a un diagnóstico de dataset, no ajustes ciegos de hiperparámetros.

■ Sobreentrenamiento

Cuando la brecha entre métricas de entrenamiento y validación crece sostenidamente, el modelo está memorizando en lugar de generalizar. El ingeniero I+D reconoce esta señal en las curvas y actúa: más regularización, early stopping, o más datos.

■ Ausencia de datos

Cuando el modelo no converge o las métricas oscilan sin estabilizarse, frecuentemente el dataset es insuficiente para la complejidad de la arquitectura. La destilación y el transfer learning son la respuesta: aprovechar conocimiento ya consolidado en modelos grandes.

■ Modelo sobredimensionado para el dataset

VGG16 con 21 ejemplos predice al azar. Ese resultado no es un fracaso del experimento — es información. Le dice al ingeniero que la arquitectura es desproporcionada y que necesita reducir capacidad o conseguir más datos.

Reflexión final

"Un ingeniero de I+D tiene que tener el conocimiento profundo de cómo funcionan las herramientas que está utilizando para poder ver caminos alternativos para lograr el éxito de su proyecto. En este curso, el objetivo nunca fue aprender a entrenar, sino generar de la forma más eficiente y eficaz un producto con valor comercial para la organización. Es por esto que las dos actividades más relevantes fueron: establecer el resultado esperado del proyecto a través de un producto, y generar una arquitectura de producto que permitiese integrar modelos de IA generados por Deep Learning, de tal forma que se pudiese actualizar los modelos sin tener que cambiar el producto."

La arquitectura de producto construida — app Ionic con canvas de captura, InferenceService como capa de abstracción, y modelos intercambiables en formato ONNX — es un patrón de ingeniería real: el producto no cambia cuando el modelo mejora. Se reemplaza el archivo .onnx y el sistema actualiza su comportamiento sin modificar una línea de código de la aplicación. Eso es valor comercial concreto: tiempo de despliegue reducido, riesgo de regresión minimizado, y capacidad de iteración sobre el modelo sin fricción en el producto.

El Deep Learning no es el destino — es el medio. El destino es un producto que resuelve un problema real, que puede mejorarse de forma independiente en cada una de sus capas, y que un equipo puede mantener y evolucionar sin depender de la persona que lo construyó originalmente.